MLOps Assignment 1
Heart Disease Risk Prediction

Machine Learning Operations (AIMLCZG523), BITS Pilani

Author: Sudarsan S

BITS ID: 2025AE05524

Group/Section: N/A

Submission date: 20 October 2026

Repository: https://github.com/Sudarsas2410/ml_ops

1. Introduction

For this assignment, I built a binary heart-disease prediction model and took it beyond a notebook: the final pipeline is served through a tested FastAPI application, packaged in Docker and deployed locally on Minikube. The input consists of 13 clinical measurements.

Original objective: Predict the presence of heart disease from 13 routine clinical measurements, and deliver the model as a reproducible, tested, containerised and monitored API.

I used the UCI Heart Disease, Cleveland subset: 303 patients, 13 features, diagnosis num (0-4) binarised to presence/absence. Binarising the original 0–4 diagnosis makes the task a binary classification problem; it does not preserve disease severity.

Repository: https://github.com/Sudarsas2410/ml_ops

Deployed API / access instructions: Local Minikube access via kubectl port-forward; see Section 8

Video: Demonstration recording to be submitted separately; outline in docs/VIDEO_SCRIPT.md.

2. Setup and installation

```
git clone https://github.com/Sudarsas2410/ml_ops && cd ml_ops
python -m venv .venv && source .venv/bin/activate
pip install -r requirements-dev.txt
python -m src.download_data && python -m src.data_processing
python -m src.eda && python -m src.train
pytest
uvicorn api.main:app --port 8000
```

Development environment: Python 3.12.9 on Rocky Linux 9.5 (x86_64), Docker Engine 28.0.1, Minikube 1.35.0 with Kubernetes 1.32.0. The API image uses python:3.13-slim.
All scripts run from a clean checkout using requirements.txt; versions are pinned.

3. Data acquisition, cleaning and EDA

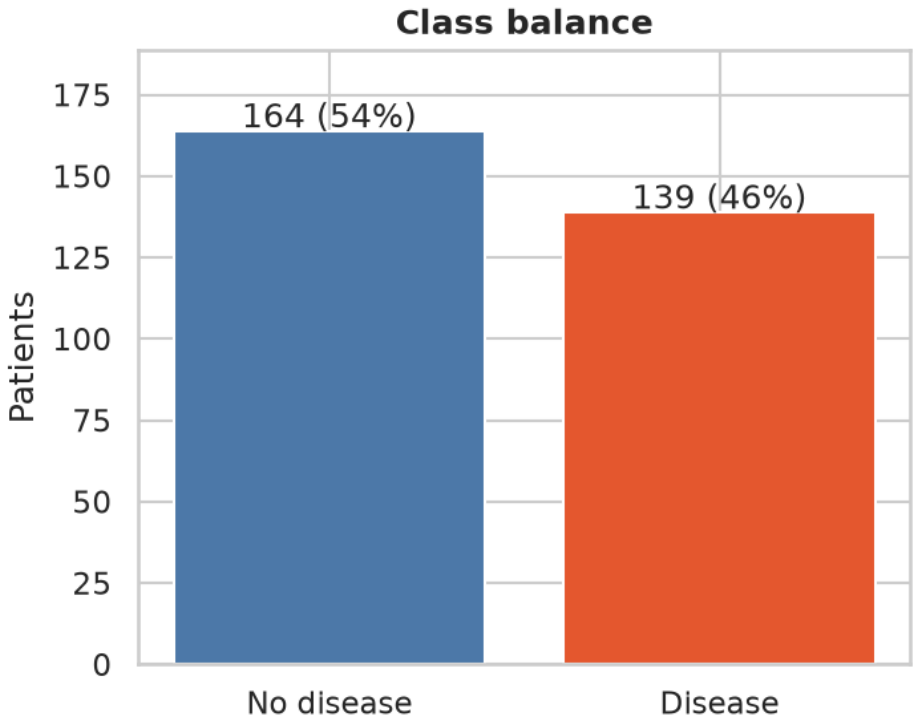
3.1 Acquisition and cleaning

I used src/download_data.py to fetch the Cleveland data from UCI, with a ucimlrepo fallback. I checked the missing-value markers before fitting the models.

Issue found	Treatment	Reason
? in ca (4 rows) and thal (2 rows)	parsed as missing, mode imputation	categorical codes
num takes values 0-4	target = (num > 0)	task requires presence/absence
Duplicate rows	0 in the cleaned dataset	No duplicate rows were found after cleaning.

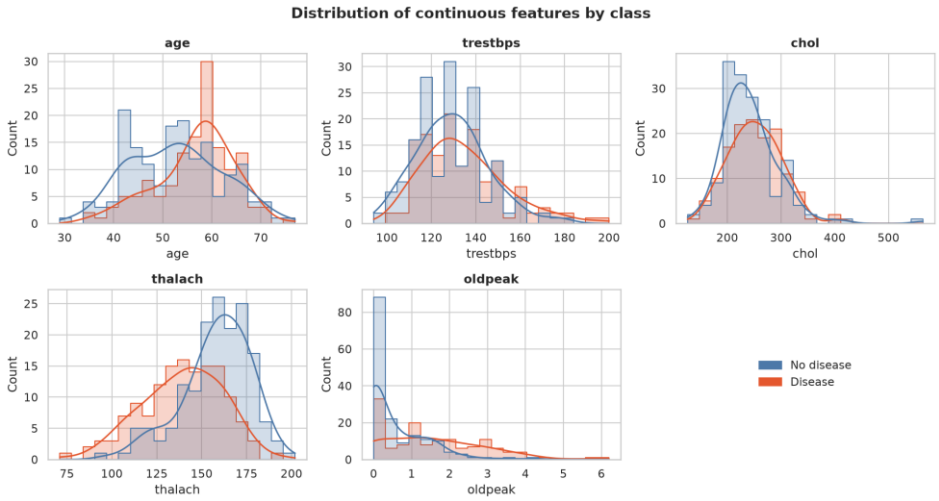
I put imputation and scaling inside the scikit-learn pipeline. This matters because each validation fold must be processed using statistics learned only from its training fold.

3.2 Exploratory analysis

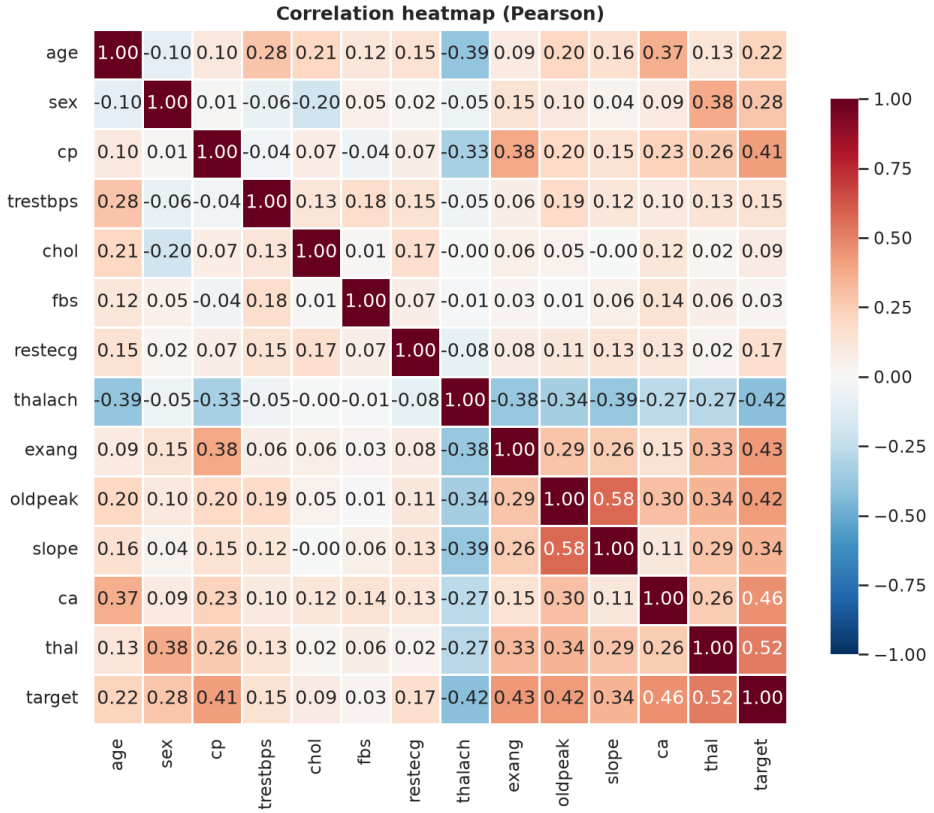


Class balance

The dataset contains 164 records without disease and 139 with disease. I therefore looked beyond accuracy and tracked both ROC-AUC and recall for the positive class.



Distributions of continuous features by class



Correlation heatmap

In the correlation plot, thal (0.522), ca (0.460), exang (0.432) and oldpeak (0.425) are positively associated with the target; thalach (-0.417) is negatively associated. I treated these as exploratory associations, not causal explanations. I did not apply an arbitrary outlier-removal rule.

4. Feature engineering and modelling choices

- Continuous features (age, trestbps, chol, thalach, oldpeak): median imputation, standardisation.
- Categorical features (sex, cp, fbs, restecg, exang, slope, ca, thal): mode imputation, one-hot encoding with handle_unknown="ignore" so unseen codes cannot break serving.
- All steps live in one ColumnTransformer inside the model Pipeline (src/data_processing.py).

I compared Logistic Regression, Random Forest and Gradient Boosting using GridSearchCV
(5-fold stratified CV, refit on ROC-AUC). Stratified 80/20 hold-out split; the model is chosen on CV ROC-AUC only.

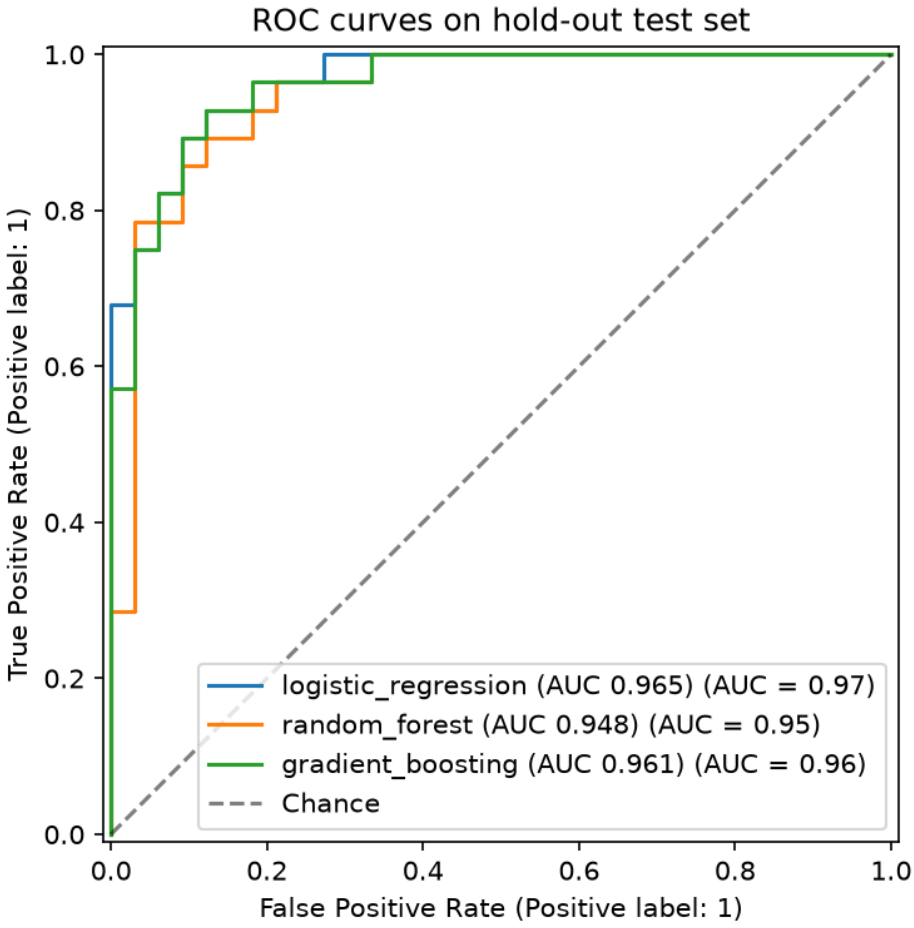
Model	Grid searched
Logistic Regression	C in {0.01, 0.1, 1, 10}; class_weight in {None, balanced}
Random Forest	n_estimators {100, 300}; max_depth {None, 4, 8}; min_samples_leaf {1, 3, 5}
Gradient Boosting	n_estimators {100, 200}; learning_rate {0.05, 0.1}; max_depth {2, 3}

I selected the model using cross-validation ROC-AUC rather than the hold-out test score. I also checked recall because missed positive cases are important in this problem. This is a coursework prototype, not a clinical diagnostic system.

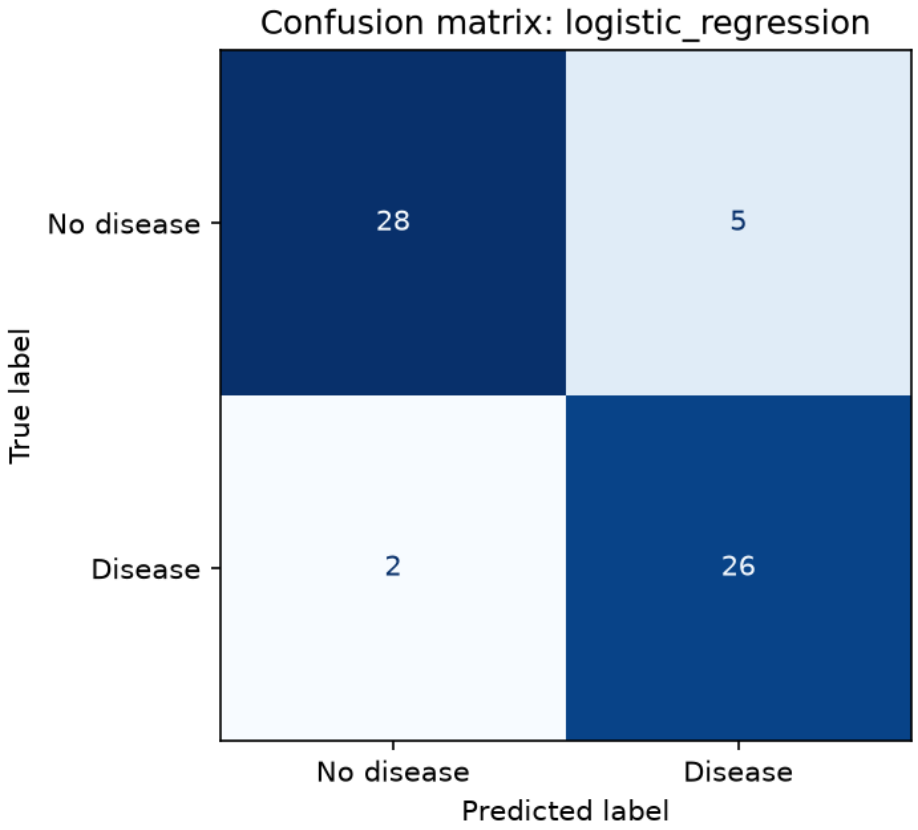
4.1 Results

I used the cross-validation column to choose the model; the remaining columns show performance on the held-out test set.

Model	CV ROC-AUC	Test accuracy	Test precision	Test recall	Test ROC-AUC
Logistic Regression	0.903 ± 0.017	0.885	0.839	0.929	0.965
Random Forest	0.902 ± 0.033	0.852	0.806	0.893	0.948
Gradient Boosting	0.874 ± 0.025	0.902	0.867	0.929	0.961



ROC curves on the hold-out set



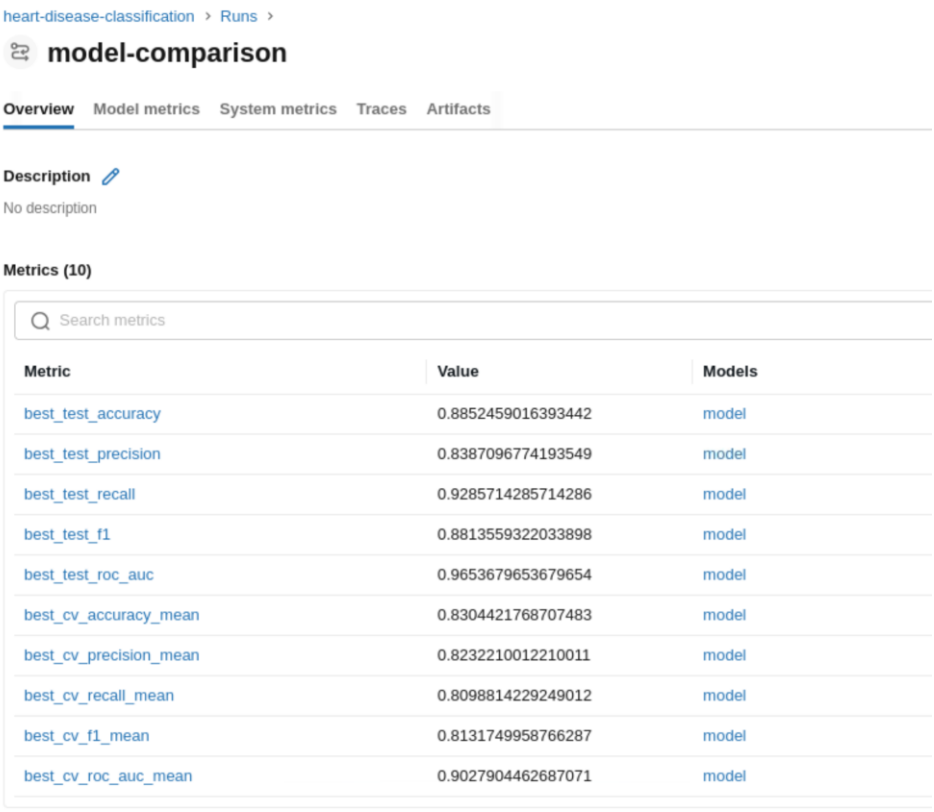
Confusion matrix of the selected model

Logistic Regression narrowly led on cross-validation ROC-AUC (0.903 ± 0.017), compared with 0.902 ± 0.033 for Random Forest. I kept Logistic Regression because that was the selection rule set before reviewing the test results. Its hold-out ROC-AUC was 0.965 and recall was 0.929. Gradient Boosting had better test accuracy (0.902 versus 0.885), which is a useful reminder that model rankings depend on the metric. The sample is small and I did not test on an independent hospital dataset.

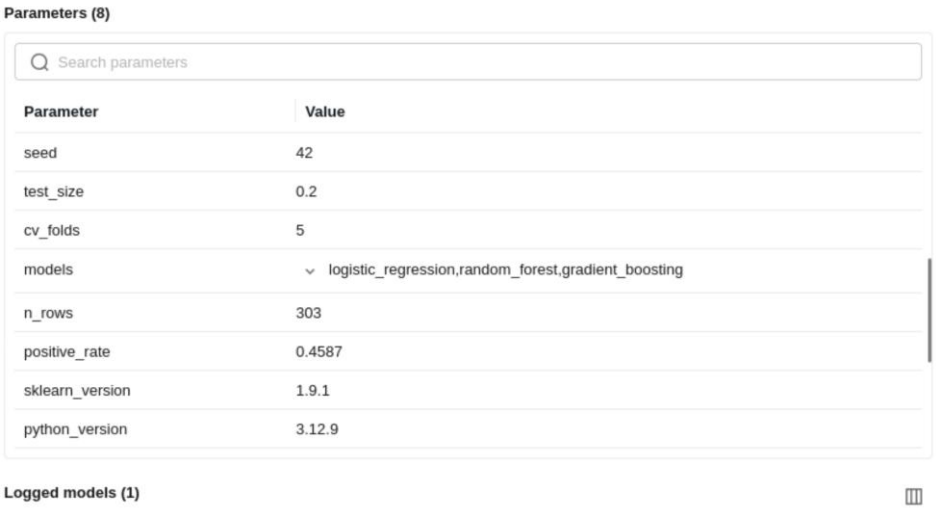
5. Experiment tracking summary (MLflow)

I used a local SQLite-backed MLflow store to record: parameters (best hyper-parameters, split sizes, seed), CV metrics (mean and std), hold-out metrics, fit time, plots (ROC, confusion matrix, feature importance, model comparison), the full grid-search table, a classification report and the fitted pipeline with signature and input example. One parent run groups the three nested model runs; the best model is registered as heart-disease-classifier.

The tracking store contains six runs. The screenshots below show the recorded metrics and parameters for the comparison run. I chose the final model from the cross-validation results, not by picking the highest hold-out accuracy.



MLflow experiment metrics



MLflow tracked parameters

6. Model packaging and reproducibility

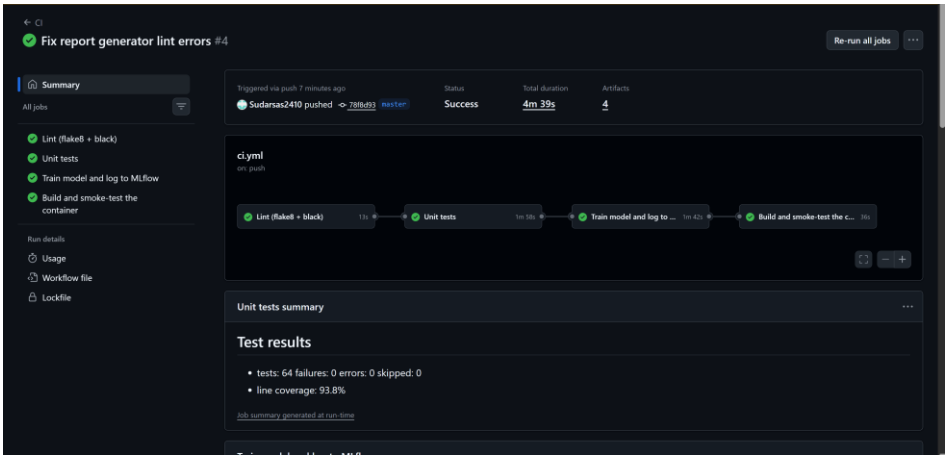
- models/model.joblib: the complete fitted pipeline (preprocessing + estimator); models/metrics.json: metrics, parameters, data checksum, library versions, timestamp.
- The same model is also logged to MLflow with a signature.
- Pinned requirements.txt (full) and requirements-serve.txt (inference only, identical numeric-stack versions).
- Fixed seed (42) for the split, CV and models; a test asserts identical predictions for identical seeds.

7. CI/CD and automated testing

The latest successful workflow ran 64 tests with approximately 94% coverage. I included tests for preprocessing, model behavior, API responses and command-line workflows.

GitHub Actions runs linting, tests, training and a Docker build/smoke test in sequence. A later stage does not run unless its prerequisites pass.

The workflow uses failing job statuses to block downstream stages. I verified a successful end-to-end run; I did not record a separate deliberate-failure demonstration.



Successful GitHub Actions CI/CD workflow

8. Containerisation and deployment

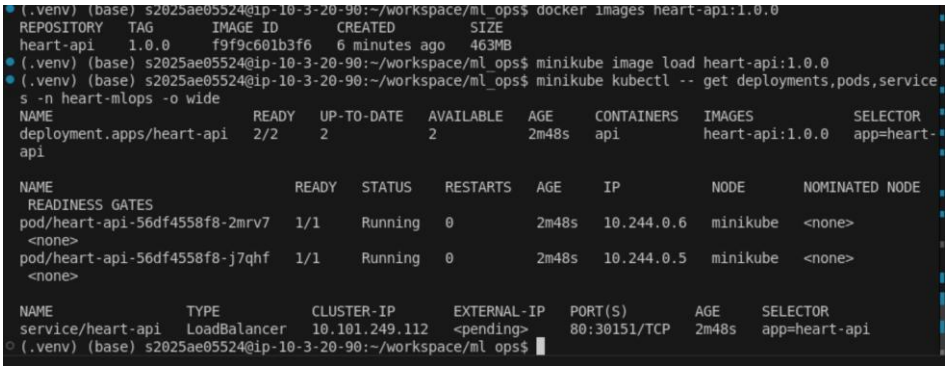
8.1 Docker

I built the API image on python:3.13-slim with serving dependencies, a non-root user, a health check and the trained model included.

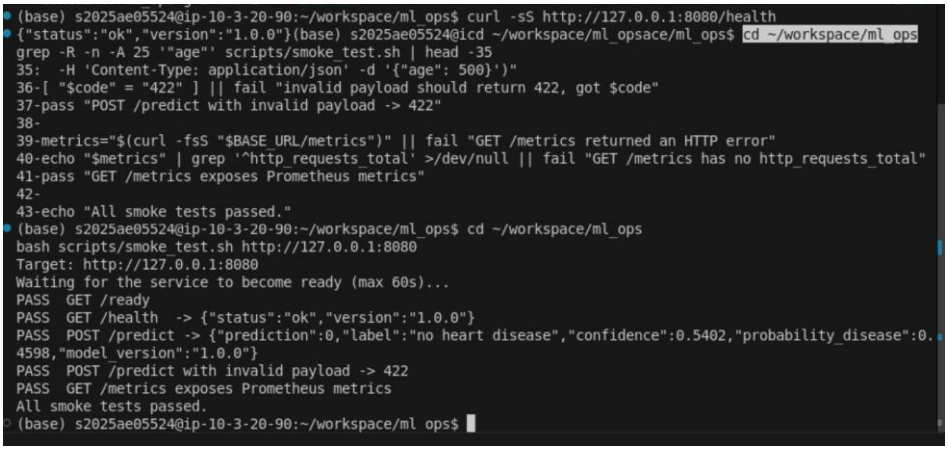
8.2 Kubernetes (Minikube v1.35.0, Kubernetes v1.32.0, Docker driver)

The Minikube deployment runs two API replicas with rolling updates, health probes and resource limits, non-root user, read-only root filesystem; LoadBalancer Service on port 80.

Access instructions: minikube kubectl -- -n heart-mlops port-forward service/heart-api 8080:80, then access http://127.0.0.1:8080/ready and /predict.



Kubernetes deployment and pod status

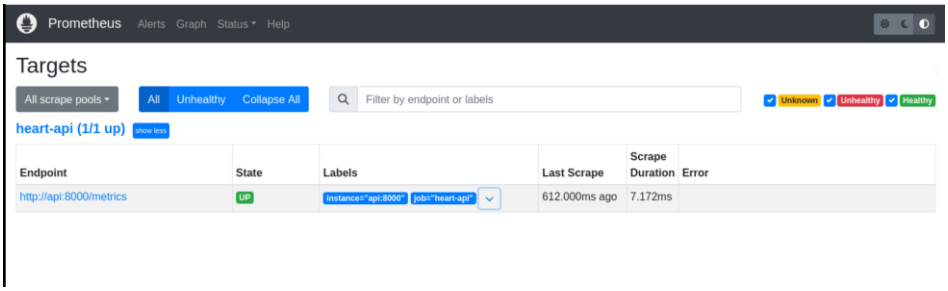


Kubernetes API smoke-test results

9. Monitoring and logging

- Structured JSON logs for every request (id, path, status, latency) and every prediction (class, confidence).
- Prometheus metrics at /metrics (request count, latency histogram, status codes, predictions by class, confidence).
- Grafana dashboard provisioned from monitoring/.

Traffic test. python scripts/generate_traffic.py --requests 300 completed with 283 successful responses (HTTP 200) and 17 invalid-payload responses (HTTP 422). The API metrics endpoint exposed request counters for these response classes, and the Prometheus target heart-api was reported as up. Exact dashboard rate and p95 latency values were not recorded, so they are not claimed here.

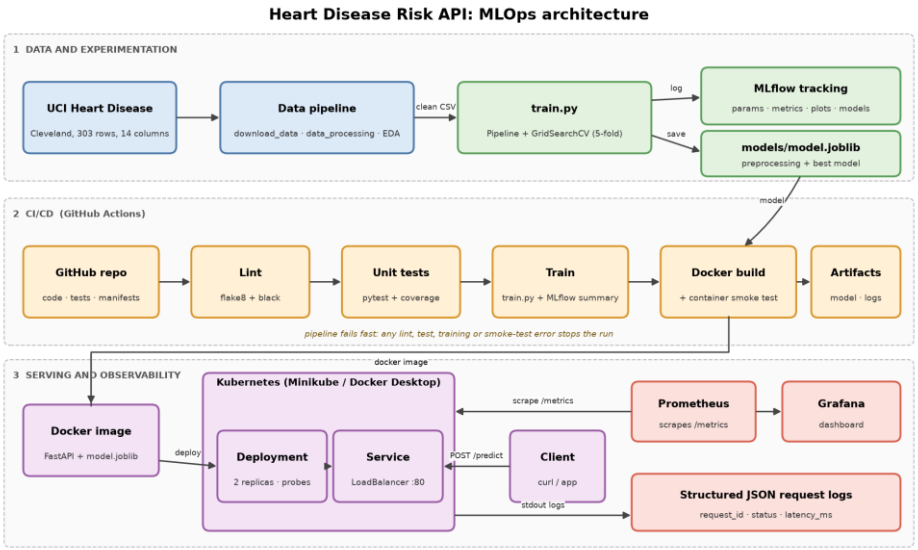


Prometheus API monitoring target



Grafana API monitoring dashboard

10. Architecture



System architecture

The architecture diagram connects the parts I tested: data preparation and model training, MLflow tracking, CI, containerized API serving, local Kubernetes deployment and monitoring. The prediction endpoint loads the saved preprocessing-and-model pipeline, while Prometheus and Grafana make API activity visible.

11.Deployment Verification and Reproducibility

Deployment verification

The heart disease prediction API was deployed locally using Minikube and Kubernetes. The deployment was configured with two replicas, and both pods were verified as ready. The API was tested through a local port-forward connection, including its readiness, prediction, and Prometheus metrics endpoints.

The deployment uses a Kubernetes Service of type LoadBalancer. In the local Minikube environment, the external IP was pending during verification, so port-forwarding was used to access the application. This is a limitation of the local environment rather than evidence of a publicly accessible endpoint.

Reproducing the project

The complete source code is available at:

https://github.com/Sudarsas2410/ml_ops

The repository contains the dataset preparation, exploratory analysis, training pipeline, MLflow experiment tracking, unit tests, Docker configuration, Kubernetes manifests, and monitoring setup. The README provides the commands and environment requirements needed to run the project.

The automated GitHub Actions workflow verifies code quality, runs tests, trains the model, and performs a Docker API smoke test. The successful workflow run is available in the repository's Actions history.

Final verification summary

Check	Result
Unit tests	64 passed
Test coverage	Approximately 94%
Kubernetes replicas	2 ready
API readiness and prediction	Verified locally
Prometheus metrics	Verified
Grafana dashboard	Verified
CI/CD workflow	Passed

Deployment note: The API is hosted in a local Minikube environment and does not have a permanent public URL. The repository contains instructions for starting the deployment and accessing the API local

12. Conclusions, limitations and future work

This exercise helped me connect model selection to the engineering work needed to run and observe an API. The strongest part of the implementation is the repeatable path from data to CI-tested container and a working local Kubernetes deployment. The main limitation is the 303-record dataset: without external validation, the results cannot be taken as clinical evidence. The API also lacks authentication and validated drift alerts. I would next test on independent data, review probability calibration and add deployment safeguards.

13. References

- Detrano, R. et al. (1989). International application of a new probability algorithm for the diagnosis of coronary artery disease. American Journal of Cardiology, 64(5), 304-310.
- Dua, D. and Graff, C. UCI Machine Learning Repository: Heart Disease. https://archive.ics.uci.edu/dataset/45/heart+disease
- scikit-learn documentation: https://scikit-learn.org/stable/ (pipeline and model-selection concepts).